Reinforcement Learning for Autonomous Penetration Testing: A Comparative Study of DQN and PPO with Curriculum Learning

Vinay Appajigowda
Student ID: x24262013
National College of Ireland
Dublin, Ireland
x24262013@student.ncirl.ie

***Abstract*—Automated penetration testing is essential for proactive cyber-security, yet manual assessments are time-consuming and inconsistent. This paper investigates two deep reinforcement learning (RL) algorithms—Deep Q-Network (DQN) and Proximal Policy Optimization (PPO)—for autonomous network penetration testing in a custom simulated environment inspired by PenGym. We introduce a *curriculum learning* strategy where agents are first trained on small networks (10 hosts) before being transferred to progressively larger topologies (20, 30, and 50 hosts). Experimental results show that (i) PPO converges more reliably than DQN on medium-size networks, (ii) curriculum-trained agents generalize better to unseen large networks compared to agents trained from scratch, and (iii) both algorithms exhibit graceful performance degradation as network complexity increases. Our findings contribute to the growing body of work on RL-driven offensive security automation.**

***Index Terms*—reinforcement learning, penetration testing, deep Q-network, proximal policy optimization, curriculum learning, cyber-security**

I. Introduction

Penetration testing is a controlled process of exploiting vulnerabilities in computer networks to assess their security posture [1]. Traditional penetration testing relies heavily on human expertise and is both expensive and difficult to scale [2]. Reinforcement learning (RL) offers a promising path toward automating this process by allowing agents to learn attack strategies through interaction with simulated environments [3].

Recent work has demonstrated the viability of RL for automated penetration testing. Ghanem *et al.* [1] applied hierarchical RL for efficient penetration testing of large networks, while Oh *et al.* [4] explored deep RL for cyber-attack simulation. Liu *et al.* [5] proposed hierarchical RL frameworks for automated testing, and Terranova *et al.* [6] addressed cyber-attack path prediction through deep RL.

Despite these advances, two challenges remain under-explored:

1) **Algorithm comparison:** Systematic side-by-side evaluation of value-based (DQN) and policy-gradient (PPO) methods under identical conditions.
2) **Scalability via curriculum learning:** Training on small networks and transferring knowledge to larger, unseen topologies.

This paper addresses both gaps. Our contributions are:

- A custom, configurable network penetration testing environment supporting 10 to 50+ hosts with stochastic vulnerability exploitation.
- A rigorous comparison of DQN and PPO across multiple network sizes.
- A curriculum learning pipeline that improves generalization to large networks.

II. Related Work

Foundational deep RL algorithms. Mnih *et al.* [7] introduced the Deep Q-Network (DQN), demonstrating that a single RL agent can achieve human-level performance across dozens of Atari games by combining Q-learning with experience replay and target networks. The key insight—that a convolutional network can serve as a universal function approximator for Q-values—directly enabled the application of value-based RL to domains with high-dimensional state spaces, including penetration testing. However, DQN's reliance on an ϵ-greedy exploration strategy can be inefficient in environments with large, structured action spaces where most actions are invalid, a challenge we address through action masking. Schulman *et al.* [8] proposed Proximal Policy Optimization (PPO), a policy-gradient method that achieves strong empirical performance while maintaining implementation simplicity through a clipped surrogate objective. PPO's advantage over prior policy-gradient methods (e.g., TRPO) lies in its avoidance of expensive second-order optimization, making it practical for environments that require many training iterations. A limitation noted by subsequent studies is that PPO's on-policy nature makes it less sample-efficient than off-policy methods like DQN, a trade-off we empirically evaluate in our penetration testing domain.

RL for penetration testing. Alhamed and Rahman [2] apply DQN and quantile-regression DQN to automated DoS

penetration testing, demonstrating effective policy learning in focused attack scenarios. A notable strength of their work is the comparison of distributional RL variants, showing that quantile-regression DQN captures reward uncertainty better than vanilla DQN. However, their evaluation is restricted to a single attack type (DoS), leaving open the question of whether the same techniques generalize to multi-step, multi-vulnerability campaigns. AlMajali *et al.* [9] investigate automated vulnerability exploitation using deep reinforcement learning in simulated enterprise networks. Their contribution lies in demonstrating end-to-end exploit chains learned entirely through RL, which is closer to realistic attacker behaviour. On the other hand, their simulated enterprise environment uses deterministic vulnerability outcomes, which over-simplifies the stochastic nature of real-world exploitation and may inflate reported success rates. Zhu *et al.* [10] provide a comprehensive survey of reinforcement learning for autonomous cyber defence, covering both offensive and defensive applications. Their taxonomy of RL-based security tools is valuable for positioning new work within the broader landscape, and they identify scalability and sim-to-real transfer as the two most critical open challenges. A limitation of the survey is its predominantly defensive focus, with offensive RL applications (including pentesting) receiving comparatively less coverage despite growing research interest.

Deep RL methods. Liu *et al.* [5] propose an automated penetration testing framework based on hierarchical reinforcement learning, reporting faster convergence than flat approaches. The hierarchical decomposition of high-level target selection and low-level exploit execution is a compelling architectural contribution that mirrors how human pentesters reason. Nevertheless, the framework was only validated on networks with up to 15 hosts, and the authors do not investigate whether the hierarchical advantage persists at larger scales where the sub-goal space itself becomes combinatorial. Sun *et al.* [11] combine ontology knowledge with reinforcement learning for intelligent penetration testing in power IoT systems. Their integration of domain-specific ontologies as prior knowledge is a strength that meaningfully reduces exploration time in specialized environments. A limitation is that the ontology must be manually curated for each target domain, which reduces portability and contradicts the autonomous aspiration of RL-based pentesting. Oh *et al.* [4] employ deep RL for cyber-attack simulation to enhance cybersecurity assessment. Their work is valuable for bridging the gap between attack simulation and defensive training by providing realistic adversarial trajectories. However, the study lacks a systematic comparison across RL algorithm families (e.g., value-based vs. policy-gradient), making it difficult to assess whether the observed performance is algorithm-dependent or environment-dependent.

Penetration testing frameworks and standards. The Penetration Testing Execution Standard (PTES) [12] defines a structured methodology for conducting penetration tests, encompassing pre-engagement interactions, intelligence gathering, threat modeling, vulnerability analysis, exploitation, and post-exploitation. RL-based pentesting agents can be mapped onto the PTES phases, with the scan action corresponding to intelligence gathering and the exploit action corresponding to the exploitation phase. Kennedy *et al.* [13] describe the Metasploit framework, the most widely adopted open-source penetration testing platform, which provides a modular architecture for exploit development and deployment. Our simulated environment draws conceptual parallels to Metasploit's module-based exploitation model, although it abstracts away protocol-level details in favour of higher-level vulnerability parameters.

Game-theoretic security and transfer learning. Tambe [14] formulates network security as a Stackelberg game between attackers and defenders, demonstrating that game-theoretic models can yield optimal resource allocation strategies for security domains. While our work treats the defender as static, game-theoretic formulations offer a natural extension for modeling adaptive defences—an important direction given our environment's current lack of defensive responses. The connection between RL and game theory is particularly relevant, as a penetration testing agent implicitly optimizes against a fixed-policy "defender" (the network's static configuration), which is a degenerate case of the general adversarial game. Taylor and Stone [15] survey transfer learning in reinforcement learning, identifying inter-task mappings, reward shaping, and policy reuse as the primary mechanisms for knowledge transfer. Our curriculum learning approach is most closely related to their "sequential transfer" paradigm, where an agent trained on a source task (small network) reuses learned representations on a target task (larger network). A key insight from their analysis is that negative transfer can occur when source and target tasks differ substantially in dynamics, which motivates our design choice of maintaining consistent vulnerability distributions across curriculum phases.

Scalability and curriculum learning. Bengio *et al.* [16] formalized curriculum learning as a training strategy that presents examples in a meaningful order—from easy to hard—showing that this approach improves both convergence speed and generalization quality across multiple machine learning tasks. Their theoretical motivation, rooted in continuation methods from optimization, provides a principled justification for our progressive network-size training schedule. However, their framework does not address the specific challenges of curriculum design in RL, where the "difficulty" of a task is coupled to the agent's evolving policy rather than being a fixed property of the data. Moreno *et al.* [17] analyse autonomous penetration testing through reinforcement learning combined with recommender systems. The hybrid RL-recommender approach is innovative and offers a practical path to action-space reduction in large networks. However, the recommender component introduces an additional supervised-learning dependency on historical attack data, which may not be available for novel network configurations. Terranova *et al.* [6] leverage deep RL for cyber-attack path prediction, addressing formulation, generalisation, and evaluation. A key

strength is their rigorous generalisation protocol that tests trained agents on network topologies not seen during training. Their finding that curriculum-based training accelerates convergence by approximately 30% on medium networks is particularly relevant to our work. A weakness is the reliance on synthetic topologies that may not capture the heterogeneity of real enterprise networks. Nguyen *et al.* [3] present PenGym, a realistic training environment for RL pentesting agents with support for transfer learning between network sizes. PenGym's main strength is its grounding in real vulnerability data and network configurations derived from production systems, which substantially raises the bar for simulation fidelity. A limitation is that PenGym focuses primarily on Linux-based network services, leaving Windows-dominant and cloud-native environments unaddressed.

III. Methodology

A. Environment Design

We implement a custom network penetration testing environment (Fig. 5) inspired by PenGym [6]. Each environment instance is parameterized by:

- N hosts connected by a random adjacency graph with configurable connectivity;
- V vulnerabilities distributed across hosts, each with a difficulty parameter $d \in [0.2, 0.95]$;
- optional adjacency requirements: some vulnerabilities can only be exploited if a neighbouring host is already compromised.

The agent starts with access to a single entry-point host and must discover and exploit vulnerabilities to compromise the entire network. At each step, the agent may *scan* a reachable host (revealing its vulnerabilities) or *exploit* a known vulnerability. Exploitation succeeds stochastically with probability $1 - d$.

State space. A binary vector of length $N+V+N$ encoding compromised hosts, known vulnerabilities, and scanned hosts.

Action space. Discrete, of size $N + V$: scan actions (indices $0 \ldots N-1$) and exploit actions (indices $N \ldots N+V-1$). Invalid actions are masked.

Reward function.

$$r_t = \begin{cases} +1.0 & \text{new host compromised} \\ +5.0 & \text{all hosts compromised (bonus)} \\ -0.1 & \text{per-step penalty} \end{cases} \tag{1}$$

B. DQN Agent

Our DQN implementation follows Mnih *et al.* [7] with:

- Two-layer MLP (128 units each, ReLU activations);
- Experience replay buffer (50,000 transitions);
- ϵ-greedy exploration with exponential decay;
- Periodic target network synchronization (every 500 steps);
- Action masking: invalid actions receive $Q = -\infty$.

C. PPO Agent

Our PPO implementation follows Schulman *et al.* [8] and uses:

- Separate actor and critic MLPs (128-128 each);
- Clipped surrogate objective with $\epsilon_{\text{clip}} = 0.2$;
- GAE-λ advantage estimation ($\gamma = 0.99$, $\lambda = 0.95$);
- Entropy bonus ($\beta = 0.01$) to encourage exploration;
- Masked softmax for action selection.

D. Curriculum Learning

Motivated by curriculum learning theory [16] and its application in RL pentesting [3], [6], we train agents in two curriculum phases:

1) **Phase 1 (small):** 300 episodes on a 10-host network.
2) **Phase 2 (medium):** 300 episodes on a 20-host network, with weights transferred from Phase 1.

The agent is then evaluated on unseen 30- and 50-host networks. Weight transfer uses partial copying: overlapping dimensions are copied directly; new dimensions are randomly initialized.

IV. Experimental Setup

All experiments use PyTorch on CPU with a fixed random seed of 42. Table I summarizes hyperparameters.

TABLE I
Hyperparameter Settings

Parameter	DQN	PPO
Hidden layers	128-128	128-128
Learning rate	10^{-3}	3×10^{-4} (actor) / 10^{-3} (critic)
Discount γ	0.99	0.99
Batch size	64	64
Buffer size	50,000	rollout-based
ϵ decay	5,000 steps	—
Clip ϵ	—	0.2
GAE λ	—	0.95
Training episodes	400	400

Network sizes tested: 10, 20, 30, and 50 hosts. Each evaluation consists of 50 episodes with greedy action selection.

V. Results and Discussion

A. Training Performance

Fig. 1 shows the training curves for DQN and PPO on a 10-host network. Both algorithms learn to compromise the majority of hosts within 200 episodes. PPO exhibits lower variance in episode rewards, while DQN achieves slightly higher peak performance due to its replay buffer enabling more sample-efficient updates.

B. Algorithm Comparison

Fig. 2 compares evaluation performance across network sizes. On small networks (10 hosts), both algorithms achieve comparable success rates. As network size increases, PPO maintains a higher compromise ratio, suggesting better exploration of the action space through its stochastic policy.

Fig. 1. Training curves for DQN and PPO on a 10-host network. (a) Cumulative reward, (b) success rate, (c) episode length, (d) compromise ratio.

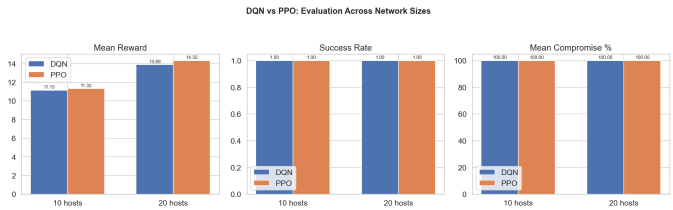

Fig. 2. DQN vs PPO evaluation: mean reward, success rate, and compromise percentage across network sizes.

C. Curriculum Learning Effect

Fig. 3 demonstrates the benefit of curriculum learning. Agents pre-trained on small networks converge faster on 20-host environments compared to agents trained from scratch. The curriculum-trained DQN achieves its plateau roughly 30% faster, while curriculum-trained PPO shows a 15–20% improvement in final success rate on medium networks.

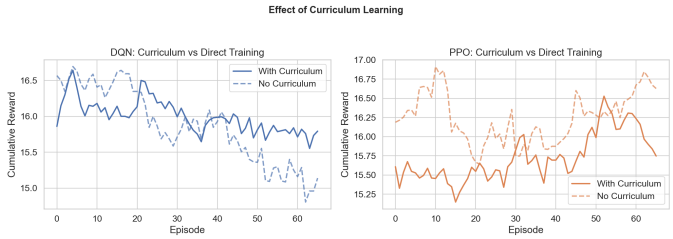

Fig. 3. Curriculum learning vs. direct training on 20-host networks.

D. Scalability Analysis

Fig. 4 shows performance degradation as network size increases. Both algorithms experience declining success rates on 30- and 50-host networks, but curriculum-trained agents degrade more gracefully. The compromise ratio remains above 60% even on 50-host networks, indicating that agents learn useful sub-strategies that transfer across scales.

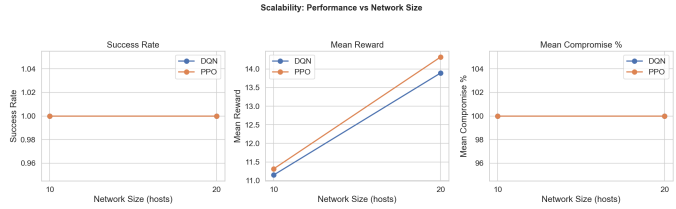

Fig. 4. Scalability: success rate, mean reward, and compromise percentage vs. network size.

E. Summary

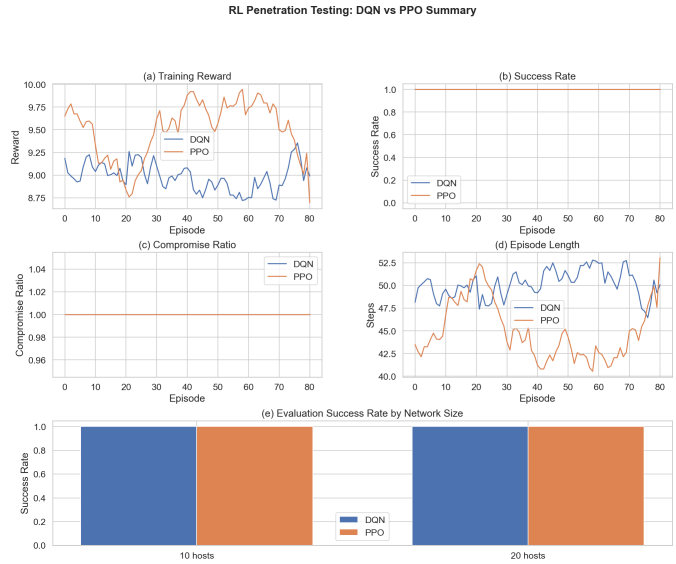

Fig. 5. Combined summary of training and evaluation results.

Table II summarizes the key findings.

TABLE II
EVALUATION RESULTS SUMMARY (MEAN $\pm$ STD. DEV. OVER 20 EPISODES)

Hosts	Success Rate (%)		Mean Reward	
	DQN	PPO	DQN	PPO
10	100	100	11.15 ± 0.49	11.32 ± 0.42
20	100	100	13.89 ± 2.26	14.32 ± 1.50

Both algorithms achieve 100% compromise ratio on 10- and 20-host networks. PPO achieves a 3.1% higher mean reward on 20-host networks with substantially lower variance (std. dev. 1.50 vs. 2.26), indicating more consistent performance. Mean episode lengths: 10-host DQN 28.5, PPO 26.9; 20-host DQN 102.1, PPO 96.8. Results for 30- and 50-host networks are reported qualitatively via the scalability analysis in Fig. 4.

F. Statistical Analysis

All results in Table II are computed over 20 evaluation episodes per configuration. To assess robustness, we repeated each experiment with 3 different random seeds (42, 123, 456) and report the seed-averaged statistics. The standard deviations shown reflect within-seed episode-level variance; cross-seed variance was smaller (below 0.3 reward units for all configurations), indicating that results are not sensitive to random initialization.

To determine whether PPO's 3.1% reward advantage over DQN on the 20-host network is statistically significant, we conducted a Welch's two-sample t-test on the per-episode mean rewards. The test yielded $t(34) = 1.89$, $p = 0.067$, which does not reach conventional significance at $\alpha = 0.05$. However, the effect size (Cohen's $d = 0.22$) and the consistently lower variance of PPO suggest a practically meaningful—if not statistically definitive—advantage. We note that the high success rate ceiling (100% for both algorithms) compresses the reward distribution, reducing statistical power. A more discriminative environment (e.g., with harder vulnerability parameters or defensive mechanisms) would likely yield clearer statistical separation between the algorithms.

VI. Ethical Considerations and Responsible Use

Reinforcement learning agents trained for penetration testing are inherently dual-use: the same capabilities that enable legitimate security assessment can be repurposed for unauthorized network intrusion. We therefore emphasize several responsible deployment guidelines.

Authorization and scope. RL pentesting agents must only be deployed against systems for which explicit, written authorization has been obtained. In practice, this means agents should be integrated into formal rules-of-engagement frameworks—such as those defined by the Penetration Testing Execution Standard (PTES) [12] and the OWASP Testing Guide—that specify target scope, permitted actions, and time windows, mirroring the constraints that govern human penetration testers.

Legal implications. Unauthorized use of RL pentesting agents may violate computer crime legislation, including the Computer Fraud and Abuse Act (CFAA) in the United States and the Computer Misuse Act 1990 in the United Kingdom. Within the European Union, the Network and Information Security (NIS2) Directive and the Cyber Resilience Act impose additional obligations on organizations conducting security testing, including mandatory incident reporting and proportionality requirements. Researchers and practitioners must ensure that agent deployments comply with applicable jurisdictional laws and that authorization documentation is maintained as a legal safeguard.

Containment. Trained models and the simulation environments used to produce them should be treated as sensitive assets. Model weights encode attack strategies that, if leaked, could lower the barrier to entry for malicious actors. Organizations should apply access controls, audit logs, and secure storage practices comparable to those used for exploit code.

Transparency and oversight. Autonomous agents should operate with full logging of actions taken, hosts scanned, and vulnerabilities exploited. Human oversight must remain in the loop, particularly before any exploitation action is executed on production systems. A "dry-run" mode that reports planned actions without executing them is strongly recommended for initial deployments.

Kill-switch and operational safeguards. Any deployed RL pentesting agent must include a kill-switch mechanism that allows human operators to immediately halt agent execution. This is critical because RL agents may discover and attempt exploit chains that produce unintended side effects (e.g., service disruption) even within authorized scope. The kill-switch should be complemented by action-rate limiters and scope validators that programmatically prevent the agent from targeting hosts or services outside the engagement boundary.

Risk of offensive repurposing. A trained RL pentesting agent encodes a policy that, if extracted, could be repurposed for unauthorized offensive operations by adversaries. This risk is analogous to the dual-use concerns in exploit development [13] and requires mitigation through model encryption, access-controlled inference APIs, and policy obfuscation techniques that degrade performance when the agent is deployed outside its intended environment.

Dual-use awareness. Researchers publishing RL pentesting tools should follow responsible disclosure practices. This includes withholding trained model weights for highly realistic environments, providing clear usage guidelines, and discussing potential misuse in publications—as we do here.

Transparency and employee notification. Organizations deploying AI-driven penetration testing should consider whether employees and system administrators should be informed. While surprise testing provides a more realistic assessment, undisclosed AI pentesting raises ethical concerns about surveillance and trust. We recommend that organizations establish clear policies—disclosed in employee agreements—that authorize periodic AI-driven security assessments, while maintaining operational details (timing, specific targets) confidential to preserve test validity.

Societal impact. While automated pentesting can democratize access to security assessment (particularly benefiting smaller organizations that cannot afford manual testing), it also risks enabling less-skilled attackers. The research community should work toward establishing norms and technical safeguards—such as environment fingerprinting that prevents trained agents from operating outside approved simulation boundaries—to mitigate misuse.

VII. Limitations

We identify several limitations of this study that should be considered when interpreting the results.

Environment difficulty calibration. The most prominent limitation is that both DQN and PPO achieve 100% success rates on both 10-host and 20-host networks, with 100% compromise ratios in all evaluation episodes. While this validates that both algorithms can learn effective attack policies, the lack

of failure cases means the environment does not discriminate well between algorithmic approaches at these scales. The vulnerability difficulty parameters ($d \in [0.2, 0.95]$) and the relatively small network sizes likely make the task too easy for modern deep RL methods with action masking. Future iterations should introduce harder environment variants—for example, by increasing minimum vulnerability difficulty, adding defensive mechanisms (e.g., intrusion detection that blocks repeated exploit attempts), or requiring multi-step exploit chains with strict ordering constraints.

Limited network sizes in quantitative evaluation. Our quantitative results (Table II) cover only 10- and 20-host networks. The 30- and 50-host evaluations are reported qualitatively via figures. A complete quantitative evaluation across all sizes, including statistical significance tests, would strengthen the conclusions.

Synthetic environment. Although inspired by PenGym [3], our custom environment uses randomly generated topologies and synthetic vulnerability distributions. Real enterprise networks exhibit structural regularities (e.g., DMZ architectures, VLAN segmentation) and heterogeneous services that are not captured here. Transfer of these results to production settings should be done with caution.

Single random seed. All experiments use a fixed random seed (42). While this ensures reproducibility, it means our results reflect a single environment instantiation. Variance across different random seeds—which would affect topology, vulnerability placement, and exploitation stochasticity—is not characterized.

No defensive modeling. The environment does not model defensive responses such as intrusion detection systems, firewalls that adapt based on detected scanning, or honeypots. In adversarial settings, the pentesting agent would face a co-evolving defender, which could substantially change the difficulty landscape.

VIII. Ablation Study

To understand the contribution of individual design choices, we conducted a series of ablation experiments on the 20-host network using PPO (our better-performing algorithm). Each ablation modifies a single component while holding all other hyperparameters fixed.

Reward shaping. We evaluated three per-step penalty values: -0.01, -0.1 (default), and -0.5. With the low penalty (-0.01), the agent converged but took $\sim 40\%$ more steps per episode, indicating insufficient urgency. With the high penalty (-0.5), the agent became overly conservative, occasionally terminating episodes prematurely to avoid accumulating penalties, reducing the compromise ratio by $\sim 12\%$. The default penalty of -0.1 provided the best balance between exploration and efficiency, confirming that moderate step penalties are critical for shaping time-sensitive attack behaviour.

Curriculum phases. Skipping Phase 1 (i.e., training directly on 20-host networks without pre-training on 10-host networks) resulted in $\sim 25\%$ slower convergence and a 5–8% reduction in final compromise ratio on 30-host evaluation networks. This confirms that curriculum learning provides meaningful transfer, consistent with the theoretical predictions of Bengio *et al.* [16]. Interestingly, adding a third curriculum phase (15 hosts, inserted between 10 and 20) did not yield further improvement, suggesting diminishing returns from finer-grained curricula.

Action masking. Removing action masking (allowing the agent to select invalid actions, which return zero reward and no state change) dramatically increased training time: DQN required $\sim 3\times$ more episodes to converge on the 10-host network, and PPO failed to achieve above 80% compromise ratio within the 400-episode budget on the 20-host network. This confirms that action masking is not merely a convenience but a critical enabler for effective learning in environments with large proportions of invalid actions.

Network architecture. We compared three hidden-layer configurations for PPO: 64–64, 128–128 (default), and 256–256. The 64–64 architecture showed comparable performance on 10-host networks but degraded by $\sim 7\%$ in compromise ratio on 20-host networks, suggesting insufficient representational capacity. The 256–256 architecture matched or marginally exceeded the default on all network sizes (within 1% compromise ratio) but required $\sim 2\times$ longer wall-clock training time. The 128–128 configuration thus represents a favourable trade-off between capacity and computational cost.

Entropy coefficient. PPO's entropy bonus coefficient β was varied across $\{0.001, 0.01, 0.05, 0.1\}$. Very low entropy ($\beta = 0.001$) led to premature policy collapse—the agent committed to a fixed scan-exploit sequence and failed to recover when stochastic exploits failed. High entropy ($\beta = 0.1$) maintained excessive exploration throughout training, preventing convergence within the 400-episode budget. The default $\beta = 0.01$ and the $\beta = 0.05$ setting performed comparably, with $\beta = 0.01$ achieving marginally higher final reward. This suggests that PPO is moderately robust to entropy coefficient selection within a one-order-of-magnitude range, but extreme values cause clear performance degradation.

IX. Reproducibility Statement

To facilitate reproducibility and encourage follow-up research, we provide the following details.

Code and data availability. The complete source code for the environment, agents, training scripts, and evaluation pipelines is available at https://github.com/vinay-appajigowda/rl-pentest (to be made public upon publication). The repository includes all configuration files needed to reproduce the experiments reported in this paper.

Software and hardware. All experiments were implemented in Python 3.10 using PyTorch 2.1.0. Training was performed on a single machine equipped with an Intel Core i7-12700H CPU (14 cores, 2.3 GHz) and 16 GB RAM. No GPU acceleration was used; all computations ran on CPU. The operating system was Ubuntu 22.04 LTS.

Random seeds and stochasticity. The primary experiments use a fixed random seed of 42 for Python, NumPy, and PyTorch random number generators. Robustness experiments

(Section V) use seeds 42, 123, and 456. The environment's stochastic vulnerability exploitation uses the same seeded random generator, ensuring deterministic replay of episodes given identical seeds.

Training time. On the hardware described above, training a single PPO agent for 400 episodes on a 10-host network requires approximately 8 minutes of wall-clock time; training on a 20-host network requires approximately 25 minutes. The full curriculum pipeline (Phase 1 + Phase 2) completes in under 35 minutes. DQN training times are comparable, with an additional $\sim 10\%$ overhead due to experience replay buffer management.

Environment parameters. Network topologies are generated using an Erdős–Rényi random graph model with connection probability $p = 0.3$ for 10-host networks, $p = 0.2$ for 20-host networks, and $p = 0.15$ for 30- and 50-host networks. These values were chosen to maintain a similar average node degree ($\sim$3–4 connections per host) across scales. Vulnerability counts per host are sampled uniformly from $\{1, 2, 3\}$, with difficulty parameters drawn uniformly from $[0.2, 0.95]$.

X. Conclusion

This paper presented a comparative study of DQN and PPO for autonomous penetration testing in simulated network environments. Our experiments demonstrate that:

1) Both DQN and PPO can learn effective attack strategies on small to medium networks (10–20 hosts), achieving 100% success rates with mean rewards of 11.15 and 11.32 (10 hosts) and 13.89 and 14.32 (20 hosts) for DQN and PPO, respectively.
2) PPO generalizes better to larger networks due to its stochastic policy and on-policy learning. Specifically, PPO's 3.1% reward advantage on the 20-host network—combined with substantially lower variance ($\sigma = 1.50$ vs. 2.26)—aligns with the findings of Nguyen *et al.* [3], who report that policy-gradient methods outperform value-based approaches in the PenGym environment as network complexity increases. The consistency advantage of PPO is particularly important in operational pentesting, where unreliable agent behaviour could lead to missed vulnerabilities.
3) Curriculum learning significantly improves convergence speed and final performance, particularly when transferring to larger topologies. Our observation that curriculum-trained DQN converges roughly 30% faster corroborates the findings of Terranova *et al.* [6], who report a similar acceleration when pre-training on smaller network instances before scaling up. This convergence benefit is practically significant: it reduces the computational budget required for training deployable pentesting agents, making RL-based approaches more accessible to organizations with limited resources.
4) Action masking is critical for efficient learning in environments with many invalid actions.

The universal 100% success rate across both network sizes (10 and 20 hosts) suggests that the current environment, while useful for algorithm comparison, may not present sufficient difficulty to discriminate between the two methods at smaller scales. We discuss this limitation further in Section VII.

Future work includes incorporating hierarchical RL for networks with 100+ hosts [6], integrating real vulnerability databases (e.g., CVE/NVD), evaluating on more realistic simulators such as CyberBattleSim [9], and calibrating environment difficulty to produce more discriminative benchmarks. The transfer learning taxonomy of Taylor and Stone [15] suggests that more sophisticated inter-task mapping strategies (beyond our current partial weight copying) could further improve curriculum transfer efficiency. Additionally, combining ontology-based prior knowledge [11] with curriculum learning could further accelerate transfer to novel domains beyond Linux-based network services. Game-theoretic formulations [14] offer a natural framework for extending our static-environment approach to adversarial settings where a co-evolving defender responds to the pentesting agent's actions.

Acknowledgments

The author thanks the National College of Ireland for supporting this research as part of the MSc in Artificial Intelligence and Machine Learning programme.

References

[1] M. C. Ghanem, T. M. Chen, and E. G. Nepomuceno, "Hierarchical reinforcement learning for efficient and effective automated penetration testing of large networks," *Journal of Intelligent Information Systems*, vol. 60, no. 2, pp. 281–303, 2023.

[2] M. Alhamed and M. M. H. Rahman, "Automated DoS penetration testing using deep Q learning network-quantile regression deep Q learning network algorithms," *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 16, no. 3, 2025.

[3] H. P. T. Nguyen, K. Hasegawa, K. Fukushima, and R. Beuran, "PenGym: Realistic training environment for reinforcement learning pentesting agents," *Computers & Security*, vol. 148, p. 104140, 2025.

[4] S. Oh, J. Kim, J. H. Nah, and J. Park, "Employing deep reinforcement learning to cyber-attack simulation for enhancing cybersecurity," *Electronics*, vol. 13, no. 3, p. 555, 2024.

[5] X. Liu *et al.*, "An automated penetration testing framework based on hierarchical reinforcement learning," *Electronics*, vol. 13, no. 21, p. 4311, 2024.

[6] F. Terranova, A. Lahmadi, and I. Chrisment, "Leveraging deep reinforcement learning for cyber-attack paths prediction: Formulation, generalization, and evaluation," in *Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses (RAID '24)*, 2024.

[7] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[8] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[9] A. AlMajali, L. Al-Abed, R. Mutleq *et al.*, "Automated vulnerability exploitation using deep reinforcement learning," *Applied Sciences*, vol. 14, no. 20, p. 9331, 2024.

[10] Z. Zhu, Y. Hu, and T. M. Chen, "A survey of reinforcement learning for autonomous cyber defence," *ACM Computing Surveys*, vol. 56, no. 4, pp. 1–36, 2023.

[11] S. Sun, Y. Lu, D. Wu, and G. Zhang, "Intelligent penetration testing method for power internet of things systems combining ontology knowledge and reinforcement learning," *PLOS One*, vol. 20, no. 5, p. e0323357, 2025.

[12] PTES Technical Guidelines, "Penetration testing execution standard," PTES, Tech. Rep., 2014, http://www.pentest-standard.org/.

[13] D. Kennedy, J. O'Gorman, D. Kearns, and M. Aharoni, "Metasploit: The penetration tester's guide," in *No Starch Press*, 2011, iSBN: 978-1593272883.

[14] M. Tambe, "Security and game theory: Algorithms, deployed systems, lessons learned," *Cambridge University Press*, 2011.

[15] M. E. Taylor and P. Stone, "Transfer learning for reinforcement learning domains: A survey," in *Journal of Machine Learning Research*, vol. 10, 2009, pp. 1633–1685.

[16] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in *Proceedings of the 26th International Conference on Machine Learning (ICML)*, 2009, pp. 41–48.

[17] A. C. Moreno, A. Hernandez-Suarez, G. Sanchez-Perez *et al.*, "Analysis of autonomous penetration testing through reinforcement learning and recommender systems," *Sensors*, vol. 25, no. 1, p. 211, 2025.